

- **MIDISTRUCT Variations — Technical Documentation**

Acrosonus Mastering · Lua script for REAPER DAW

- ***Table of Contents***

1. Overview
 2. Architecture
 3. Execution Flow
 4. The Madness Dial
 5. CFG Parameters Reference
 6. PPQ Resolution & Absolute Time Conversion
 7. Algorithm Reference
 - Syncopation
 - Smart Velocity
 - Octave & Simplify
 - Strummer
 - Gate / Staccato
 - Power Harmonizer
 - Ghost Delay
 - Melodic Mirror
 - Retrograde
 - Ratchet / Stutter
 - Reich Phasing
 - Chord Shatter
 8. MIDI I/O
 9. Fixed Item Lanes
 10. ExtState Persistence
 11. Randomization
 12. Undo Block
 13. Known Limitations
 14. Glossary
-

- ***1. Overview***

MIDISTRUCT Variations analyzes the active MIDI take of a selected item and generates **12 distinct musical variations**, each placed in its own Fixed Lane on the same REAPER track (REAPER 7+).

Key design decisions:

- **Non-destructive: Lane 0 (original) is never written to**
 - **PPQ-agnostic: Absolute time conversion handles imported MIDI files at any resolution**
 - **Single control surface: The Madness Dial (1–10) scales all 12 algorithms simultaneously**
 - **REAPER 7 native: Uses `I_FIXEDLANE` API for clean lane management**
-

• **2. Architecture**

```
CFG table
└─ apply_madness(level)      — scales all parameters via linear
  interpolation

read_notes(src_take)         — MIDI_GetNote loop → Lua note table
get_ppq_per_beat(take)       — tempo-map-aware PPQ derivation

for each variation def:
  CreateNewMIDIItemInProj()  — new item with valid MIDI source
  I_FIXEDLANE = i            — assign to lane
  PPQ conversion loop        — src PPQ → proj_time → target PPQ
  def.fn(mapped_notes, ppqbeat, item_len_ppq) — apply algorithm
  sentinel note              — anchor MIDI source length
  write_notes()              — MIDI_DisableSort + InsertNote loop +
MIDI_Sort
  D_LENGTH = item_len        — lock item length
  B_LOOPSRC = 0              — disable auto-resize

I_NUMFIXEDLANES = 13         — expose all lanes on track
Undo_EndBlock()
```

The defs table is the extension point — adding a new variation requires only a new entry with a name and fn field.

• **3. Execution Flow**

```
math.randomseed()           high-entropy seed: os.time() + time_precise()
× 100000

load_last_madness()          GetExtState → pre-fill dialog
ask_madness_level()          GetUserInputs dialog
save_last_madness()          SetExtState (persist = true)
apply_madness(level)         interpolate all CFG parameters

GetSelectedItem(0,0)         validate item
GetActiveTake() + TakeIsMIDI() validate take
read_notes(src_take)         MIDI_GetNote loop → src_notes table
                             exit if empty
```

```

Undo_BeginBlock()

SetMediaTrackInfo(I_NUMFIXEDLANES, 13)
SetMediaItemInfo(item, I_FIXEDLANE, 0)    pin original to lane 0

for i, def in ipairs(defs):
    CreateNewMIDIItemInProj(track, item_pos, item_pos + item_len)
    SetMediaItemInfo(new_item, I_FIXEDLANE, i)
    SetMediaItemInfo(new_item, B_LOOPSRC, 0)
    GetSetMediaItemTakeInfo_String(P_NAME, ...)

    for each src_note:
        proj_time = MIDI_GetProjTimeFromPPQPos(src_take, note.ppq)
        m_ppq      = MIDI_GetPPQPosFromProjTime(new_take, proj_time)
        → mapped_notes table

    new_ppqbeat      = get_ppq_per_beat(new_take)
    new_item_len_ppq = get_item_length_ppq(new_take, new_item)
    sentinel_ppq      = max_end_ppq + new_ppqbeat

    notes = def.fn(mapped_notes, new_ppqbeat, new_item_len_ppq)
    notes[#notes+1] = sentinel note (muted, pitch=0, vel=1)
    write_notes(new_take, notes, cc_evts)

    SetMediaItemInfo(new_item, D_LENGTH, item_len)
    UpdateItemInProject(new_item)

Undo_EndBlock()
UpdateArrange()

```

• **4. The Madness Dial**

apply_madness(level) normalizes the 1–10 input to $t = (\text{level}-1)/9.0$ and linearly interpolates each CFG parameter:

```

local function dial(calm, chaos) return calm + (chaos - calm) * t end

CFG.SYNC_CHANCE      = dial(0.10, 0.80)
CFG.GATE_DURATION_RATIO = dial(0.40, 0.08)
CFG.GHOST_REPEATS     = floor(dial(2, 5))
CFG.GHOST_PITCH_DRIFT  = (t >= 0.6) and 1 or 0    -- step function at level 6
CFG.RETRO_PORTION      = dial(0.15, 1.0)
CFG.RATCH_DIVISIONS    = floor(dial(2, 8))
CFG.PHASE_GRID_BEATS   = floor(dial(7, 3))        -- decreasing: more
dissonant at high M

```

• **5. CFG Parameters Reference**

- **Syncopation**

Key	Default	Calm (L1)	Chaos (L10)	Description
SYNC_CHANCE	0.30	0.10	0.80	Fraction of notes displaced by half-beat
SYNC_HUMANIZE_MAX	10	3	30	±tick drift on displaced notes
SYNC_MICRO_MAX	3	1	15	±tick micro-drift on untouched notes

- **Smart Velocity**

Key	Default	Calm	Chaos	Description
VEL_DOWNBEAT_BOOST	1.15	1.05	1.40	Velocity multiplier on downbeats
VEL_OFFBEAT_CUT	0.80	0.92	0.50	Velocity multiplier on off-beats
VEL_ACCENT_SWING	0.08	0.03	0.20	Sinusoidal swell depth
VEL_DOWNBEAT_NEAR	0.08	—	—	Beat-phase window classified as downbeat
VEL_HUMANIZE_MAX	4	2	15	±velocity units of humanization noise

- **Octave & Simplify**

Key	Default	Calm	Chaos	Description
SIMP_REMOVE_CHANCE	0.20	0.05	0.55	Base removal probability
SIMP_SHORT_BIAS	1.6	1.2	3.0	Extra removal weight for short notes
SIMP_OCTAVE_CHANCE	0.15	0.05	0.45	Fraction of survivors shifted ±1 octave
SIMP_LEGATO_GLUE	0.92	0.99	0.70	Stretch ratio toward next note

- **Strummer**

Key	Default	Calm	Chaos	Description
STRUM_TIC	15	8	40	Ticks between each

Key	Default	Calm	Chaos	Description
K_OFFSET				strum step
STRUM_VE L_DECAY	0.92	0.98	0.72	Velocity multiplier per step
STRUM_HU MAN_MAX	2	1	6	±tick organic jitter per step
STRUM_DIR ECTION	“alternate ”	—	—	“up” / “down” / “alternate”

- **Gate / Staccato**

Key	Default	Calm	Chaos	Description
GATE_DUR ATION_RAT IO	0.25	0.40	0.08	Note length as fraction of original
GATE_VEL_ BOOST	1.20	1.10	1.45	Velocity compensation multiplier
GATE_MIN_ TICKS	20	—	—	Minimum note length floor (ticks)

- **Power Harmonizer**

Key	Default	Description
HARM_FIFTH SEMIS	7	Upper harmony interval (semitones)
HARM_SUB_ SEMIS	-12	Sub-bass interval (semitones)
HARM_FIFTH VEL_MULT	0.80	Fifth layer velocity ratio (→1.00 at L10)
HARM_SUB_ VEL_MULT	0.70	Sub-bass layer velocity ratio (→0.92 at L10)
CC_RISER_N UM	74	CC number for filter riser
CC_RISER_ST EPS	32	Riser resolution (events)

- **Ghost Delay**

Key	Default	Calm	Chaos	Description
GHOST_REPE ATS	3	2	5	Echo copies per note
GHOST INTE	2	—	—	Interval = ppqbeat / N

Key	Default	Calm	Chaos	Description
RVAL_DIV				
GHOST_VEL_DECAY	0.50	0.60	0.35	Velocity multiplier per echo
GHOST_PITCH_DRIFT	0	0	1	Semitones per echo (step at L6)

- **Retrograde**

Key	Default	Calm	Chaos	Description
RETRO_PORTION	0.50	0.15	1.0	Fraction of item reversed (from end)

- **Ratchet / Stutter**

Key	Default	Calm	Chaos	Description
RATCH_CHANCE	0.20	0.08	0.55	Probability a long note is ratcheted
RATCH_DIVISIONS	4	2	8	Sub-note count
RATCH_VEL_DECAY	0.85	0.95	0.65	Velocity multiplier per step
RATCH_LENGTH_BIAS	1.5	1.2	3.0	Length-to-probability multiplier

- **Reich Phasing**

Key	Default	Calm	Chaos	Description
PHASE_GRID_BEATS	7	7	3	Phase cycle length in beats
PHASE_OFFSET_TICKS	10	4	24	Tick nudge per beat within cycle

- **Chord Shatter**

Key	Default	Calm	Chaos	Description
SHAT_CHANCE	0.70	0.30	0.95	Probability a chord is shattered
SHAT_STEP_DIV	2	2	8	Step size = ppqbeat / N
SHAT_OCTAVE_CHANCE	0.20	0.05	0.50	Octave jump probability (non-bass notes)

Key	Default	Calm	Chaos	Description
SHAT_VEL_DECAY	0.90	0.97	0.70	Velocity decay per step

- **6. PPQ Resolution & Absolute Time Conversion**

- **The Problem**

REAPER's CreateNewMIDIItemInProj always creates items at the project's native PPQ (960 by default). Imported .mid files may have a different internal resolution (commonly 480 PPQ, sometimes 96 or 240 PPQ). Copying raw PPQ values directly compresses all notes toward the item start.

Example:

```
Source .mid:  480 PPQ  →  1 quarter note = 480 ticks
New item:     960 PPQ  →  1 quarter note = 960 ticks
```

Direct copy of 480 ticks → plays back as half a beat (wrong)

- **The Fix**

Every note position is converted through **absolute project time** as a bridge:

```
for _, n in ipairs(src_notes) do
  -- Step 1: source PPQ → absolute time in seconds
  local proj_start = reaper.MIDI_GetProjTimeFromPPQPos(src_take, n.startppq)
  local proj_end   = reaper.MIDI_GetProjTimeFromPPQPos(src_take, n.endppq)

  -- Step 2: absolute time → target PPQ (new item's grid)
  local m_start = math.floor(reaper.MIDI_GetPPQPosFromProjTime(new_take,
    proj_start) + 0.5)
  local m_end   = math.floor(reaper.MIDI_GetPPQPosFromProjTime(new_take,
    proj_end)   + 0.5)

  -- Guard: minimum 1 tick duration
  if m_end <= m_start then m_end = m_start + 1 end
end
```

This is mathematically correct regardless of:

- Source file PPQ (480, 96, 240, 960, etc.)
- Tempo map complexity (constant, ramped, marker-based)
- Item position within the project

- **Sentinel Note**

After writing variation notes, a muted silent note is inserted at `sentinel_ppq = max_end_ppq + ppqbeat` (1 beat past the last real note). This anchors the MIDI source length so REAPER does not auto-truncate the item when the variation has fewer or shorter notes than the original:

```
notes[#notes + 1] = {
    selected = false, muted = true,
    startppq = sentinel_ppq - 1,
    endppq    = sentinel_ppq,
    chan = 0, pitch = 0, vel = 1,
}
```

- **7. Algorithm Reference**

- **7.1 Syncopation**

```
half_beat = ppqbeat / 2
```

```
for each note:
    if chance(SYNC_CHANCE):
        new_start = max(0, note.start + half_beat + rand(±SYNC_HUMANIZE_MAX))
        new_start = min(new_start, item_len_ppq - 1)  -- boundary guard
        note.end   = min(new_start + dur, item_len_ppq)
    else:
        note.start = max(0, note.start + rand(±SYNC_MICRO_MAX))
        note.end   = note.start + dur
```

```
re-sort by startppq
```

- **7.2 Smart Velocity**

```
phase = (note.start % ppqbeat) / ppqbeat  -- 0.0 → 1.0
```

```
if near_downbeat(phase):
    vel *= VEL_DOWNBEAT_BOOST
else:
    vel *= VEL_OFFBEAT_CUT
```

```
swell = 1 + VEL_ACCENT_SWING * cos(phase * 2π)
vel   = clamp(vel * swell + rand(±VEL_HUMANIZE_MAX), 1, 127)
```

- **7.3 Octave & Simplify**

Pass 1 — Weighted removal:

```
len_ratio = note_length / median_length
remove_prob = min(SIMP_REMOVE_CHANCE * (SIMP_SHORT_BIAS / max(len_ratio, 0.1)), 0.85)
```

Pass 2 — Directional octave shift:


```

dir = (note.pitch < 60) and +1 or -1
if chance(0.10): dir = -dir
note.pitch = clamp(note.pitch + dir * 12, 0, 127)

```

Pass 3 — Legato-glue with same-pitch guard:

```

new_end = note.start + floor(gap_to_next * SIMP_LEGATO_GLUE)
if cur.pitch == next.pitch:
    new_end = min(new_end, next.start - 2)    -- 2-tick gap prevents VST
collision

```

• 7.4 Strummer

Groups notes within ± 2 ticks. Sorts by pitch (alternating direction per group). Staggers by STRUM_TICK_OFFSET + jitter ticks per step. Discards steps beyond item_len_ppq.

```

tick_shift = max((step - 1) * STRUM_TICK_OFFSET + rand( $\pm$ STRUM_HUMAN_MAX), 0)
vel_scale = STRUM_VEL_DECAY ^ (step - 1)

```

• 7.5 Gate / Staccato

```

new_dur = max(floor(orig_dur * GATE_DURATION_RATIO), GATE_MIN_TICKS)
note.end = note.start + new_dur
note.vel = clamp(note.vel * GATE_VEL_BOOST, 1, 127)

```

• 7.6 Power Harmonizer

Scale detection:

```

scale[note.pitch % 12] = true    -- build pitch-class set from all source
notes

```

Diatonic snap:

```

function snap_to_scale(pitch, scale):
    if scale[pitch % 12]: return pitch
    for delta = 1..6:
        if scale[(pitch + delta) % 12]: return pitch + delta
        if scale[(pitch - delta) % 12]: return pitch - delta
    return pitch    -- fallback

```

Unison guard: if snap collapses onto the original pitch, the fifth layer is skipped.

CC Riser (smoothstep cubic, written after notes):

```

smooth_t = t2 × (3 - 2t)
cc_val = CC_RISER_START + range × smooth_t

```

• 7.7 Ghost Delay

```

interval = floor(ppqbeat / GHOST_INTERVAL_DIV)

```

```

for rep = 1..GHOST_REPEATS:
    echo_start = note.start + interval × rep

```

```

if echo_start >= item_len_ppq: break      -- boundary test BEFORE decay
vel      = vel * GHOST_VEL_DECAY
acc_drift = acc_drift + GHOST_PITCH_DRIFT
echo_dur  = max(floor(orig_dur * 0.9^rep), 20)
echo_dur  = min(echo_dur, item_len_ppq - echo_start)

```

- **7.8 Melodic Mirror**

```

axis      = median(all pitches)             -- or 60 in "center" mode
mirrored = 2 * axis - note.pitch

```

```

-- Octave fold (preserves interval identity vs hard clamp)
while mirrored < 0: mirrored += 12
while mirrored > 127: mirrored -= 12

```

Deduplication: notes sharing the same **startppq** and **pitch** after fold are removed (fold can collapse distinct pitches onto the same class).

- **7.9 Retrograde**

```

cutoff = floor(item_len_ppq * (1 - RETRO_PORTION))

```

```

for each note:
    if note.start < cutoff:
        pass through unchanged
    else:
        new_start = cutoff + (item_len_ppq - note.end)
        new_end   = new_start + dur
        clamp both to [cutoff, item_len_ppq]

```

Notes inside the window are mirrored around the midpoint of [cutoff, item_len_ppq]. Notes outside pass through unchanged for smooth boundary transitions.

- **7.10 Ratchet / Stutter**

```

len_ratio = note_duration / median_length
ratch_prob = min(RATCH_CHANCE * len_ratio * RATCH_LONG_BIAS,
0.85×RATCH_CHANCE×4)

if note_dur < ppqbeat/4 or not chance(ratch_prob):
    pass through

else:
    sub_dur = floor(dur / RATCH_DIVISIONS)
    for step = 0..RATCH_DIVISIONS-1:
        sub_start = note.start + step * sub_dur
        sub_end   = sub_start + sub_dur    (last step fills to note.end)
        vel_scale = RATCH_VEL_DECAY ^ step

```

Notes shorter than ppqbeat/4 are never ratcheted (prevents inaudible sub-divisions).

- **7.11 Reich Phasing**

```
grid_ppq      = ppqbeat * PHASE_GRID_BEATS

for each note:
  cycle       = floor(note.start / grid_ppq)
  pos_in_grid = note.start % grid_ppq
  beat_in_cycle = floor(pos_in_grid / ppqbeat)
  nudge       = beat_in_cycle * PHASE_OFFSET_TICKS
  new_start   = cycle * grid_ppq + pos_in_grid + nudge
  new_start   = new_start % item_len_ppq -- wrap at item boundary
```

The micro-nudge ($\text{PHASE_OFFSET_TICKS} \times \text{beat_in_cycle}$) models Reich's tape acceleration: notes in later beats within a cycle are pushed slightly forward, creating a sense of *accelerando* within each phase cycle.

- **7.12 Chord Shatter**

```
step_ppq = max(floor(ppqbeat / SHAT_STEP_DIV), 10)

for each chord group (same as Strummer grouping):
  if #group <= 1 or not chance(SHAT_CHANCE):
    pass through all notes

  else:
    sort by pitch (low → high: bass stays on beat)
    for step, note in group:
      new_start = group.anchor + (step-1) * step_ppq
      if new_start >= item_len_ppq: break
      if step > 1 and chance(SHAT_OCTAVE_CHANCE):
        pitch += 12 (if in range 0-127)
        vel_scale = SHAT_VEL_DECAY ^ (step-1)
```

- **8. *MIDI I/O***

- **Reading Notes**

```
local notes, idx = {}, 0
while true do
  local ok, sel, mut, s, e, ch, p, v = reaper.MIDI_GetNote(take, idx)
  if not ok then break end
  notes[#notes+1] = { selected=sel, muted=mut, startppq=s, endppq=e,
                     chan=ch, pitch=p, vel=v }
  idx = idx + 1
end
```

- **Writing Notes**

MIDI_DisableSort before the first insert prevents $O(n^2)$ internal re-sorting. MIDI_Sort is called once after all inserts.

```
reaper.MIDI_DisableSort(take)
for _, n in ipairs(notes) do
    reaper.MIDI_InsertNote(take, n.selected, n.muted,
        n.startppq, n.endppq, n.chan, n.pitch, n.vel, false)
end
reaper.MIDI_Sort(take)
```

- **PPQ-per-Beat Derivation**

Derived from the tempo map via round-trip conversion rather than assuming 960:

```
local pos0 = reaper.MIDI_GetProjTimeFromPPQPos(take, 0)
local beats = reaper.TimeMap2_timeToBeats(0, pos0)
local tlbeat = reaper.TimeMap2_beatsToTime(0, beats + 1, nil)
local ppq1 = reaper.MIDI_GetPPQPosFromProjTime(take, tlbeat)
if ppq1 <= 0 then ppq1 = 960 end -- fallback
```

- **Why CreateNewMIDIItemInProj — Not AddTakeToMediaItem**

AddTakeToMediaItem creates a take with no PCM source object attached. Both MIDI_InsertNote and MIDI_SetAllEvs return false silently on sourceless takes (confirmed on REAPER 7.71/x64). CreateNewMIDIItemInProj creates a fully-initialized MIDI source that accepts writes immediately.

- **9. Fixed Item Lanes**

```
-- Enable Fixed Lanes on the track (1 original + 12 variations)
reaper.SetMediaTrackInfo_Value(track, "I_NUMFIXEDLANES", 13)

-- Pin original to lane 0
reaper.SetMediaItemInfo_Value(item, "I_FIXEDLANE", 0)

-- Assign each variation to its lane
for i, def in ipairs(defs) do
    reaper.SetMediaItemInfo_Value(new_item, "I_FIXEDLANE", i)
end
```

I_NUMFIXEDLANES is a REAPER 7+ track property. Setting it to N makes the track display N stacked lanes. Each item's I_FIXEDLANE (0-based) determines which lane it occupies.

- **10. ExtState Persistence**

The last-used Madness level is persisted across REAPER sessions:

```
-- Write (persist = true → survives REAPER restart, stored in reaper.ini)
```

```

reaper.SetExtState("AcrosonusMastering", "MIDISTRUCT_madness",
tostring(level), true)

-- Read
local val = reaper.GetExtState("AcrosonusMastering", "MIDISTRUCT_madness")
local n = tonumber(val)
if n and n >= 1 and n <= 10 then return math.floor(n) end
return 5 -- factory default

```

• **11. Randomization**

```

-- High-entropy seed: second-precision + sub-millisecond component
math.randomseed(os.time() + math.floor(reaper.time_precise() * 100000))

-- Warm up the LCG state (discards poorly-distributed initial outputs)
for _ = 1, 10 do math.random() end

```

Two runs at the same Madness level always produce different results. There is no mechanism to reproduce an exact output — intentional design for creative exploration.

• **12. Undo Block**

All REAPER write operations are wrapped in a single undo block. Ctrl+Z removes all 12 generated lanes atomically.

```

reaper.Undo_BeginBlock()
-- ... all CreateNewMIDIItemInProj, InsertNote, InsertCC calls ...
reaper.Undo_EndBlock(
    string.format("MIDISTRUCT Variations v4 - 12 fixed lanes, Madness %d",
madness), -1)

```

The -1 flag tells REAPER to auto-detect the undo scope from operations within the block.

• **13. Known Limitations**

- Only the first selected item is processed — no batch mode
 - MIDI CC, pitch-bend, and aftertouch from the source are not copied to variations (notes only), except for the CC74 riser in Lane 6
 - Reich Phasing wraps notes at item_len_ppq — notes near the end of a long item may wrap to position 0, which is musically acceptable but worth noting
 - Ratchet subdivisions are evenly spaced — no swing grid
 - Fixed Lanes require REAPER 7.0+. On earlier versions the items will overlap instead of lane-stacking
-

• **14. Glossary**

Term	Definition
PPQ	Pulses Per Quarter note — internal timing resolution of a MIDI source
Downbeat	Note whose beat phase is within VEL_DOWNBEAT_NEAR of a beat boundary
Smoothstep	Cubic interpolation $t^2(3-2t)$ — eases in and out naturally
Deep copy	Full table duplication with no shared references to the original
Legato-glue	Extending a note's end toward the next note's start to reduce silence gaps
ExtState	REAPER's persistent key-value store, written to reaper.ini
Octave fold	Shifting an out-of-range pitch by ± 12 until it falls within 0–127
Pitch drift	Accumulated semitone offset per echo in Ghost Delay
Retrograde	Time-reversal of a melodic phrase (Bach/Messiaen technique)
Ratchet	Subdivision of a long note into rapid-fire equal sub-notes
Polymetry	Simultaneous use of two or more conflicting metric patterns
Fixed Lane	REAPER 7 item display mode — items stacked vertically on one track
Sentinel note	Muted silent note at item_len_ppq used to anchor MIDI source length
Beat phase	Note position within its beat, normalized to 0.0–1.0
LCM	Least Common Multiple — beats before two polymetric patterns realign

MIDISTRUCT Variations — © 2026 Acrosonus Mastering. All rights reserved. Copyright (C) 2026 Acrosonus Mastering

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.